SynthProof, explained from scratch

For the four people who have to defend it

You are about to present a project about differential privacy. If you do not yet know what that phrase means, this document is for you. It assumes you know what a database is and what a model is, and nothing else.

Read it once end to end. Then read section 7 twice, because that is the hardest idea in the project and the one a professor is most likely to push on. Everything here is written so you can explain it in your own words afterwards — that is the only test that matters.

Every number in this document comes from a file in the repository. Where a number appears, the file it came from is named next to it. If you are asked "where did that come from", the answer is always a path you can open.

If you have twenty minutes and a panel outside the door, read section 12 — the cheat sheet — then section 7. Those two carry the project. **If you have an hour**, read the whole thing once and then drill section 11 out loud with someone else asking the questions.

What is in here

1. The problem, in plain language
2. What differential privacy actually is
3. Synthetic data, and why "it's fake so it's safe" is false
4. Our four generators, one paragraph each
5. The accountant and the ledger
6. Auditing: attacking our own release on purpose
7. The ceiling — the hardest idea, and the one you must own
8. The confound, in plain language — RETRACTED
9. Our results, and what they do NOT mean
10. Glossary
11. The Q&A drill
12. Cheat sheet — one page, take it in with you

1. The problem, in plain language

A hospital has records for fifty thousand patients. Somewhere in that table is the answer to a real question — which treatment works better, which patients get missed, where the system fails people. A researcher wants to look. The hospital cannot let them, because the table is full of real people who did not agree to be looked at by a stranger.

So the hospital does the obvious thing. It deletes the names. It deletes the addresses. It sends what is left and calls it anonymous.

This does not work, and we have known it does not work since 1997.

The story to tell

In 1997 the state of Massachusetts released hospital records for its employees. It removed names and addresses first, and the governor publicly reassured people that privacy was protected.

A graduate student named Latanya Sweeney bought the Cambridge voter roll for twenty dollars. It listed name, address, ZIP code, birth date and sex. The hospital release still contained ZIP code, birth date and sex — those did not look like identifying information, so nobody had removed them.

She matched the two lists. Exactly one person in the voter roll had the governor's ZIP code, birth date and sex. She mailed his own medical records to his office.

Sweeney later measured how general the trick was. **About 87% of Americans are uniquely identified by ZIP code, birth date and sex alone** — three fields nobody thinks of as a name.

Why it fails, in one sentence

Removing a name does not remove a person. The *combination* of the columns you kept is itself a fingerprint, and someone with a second list can match it.

And it is not just old-fashioned anonymisation

You might think modern tools solve this. In 2022 a team at EPFL tested the popular synthetic data generators properly — attacking them the way an actual adversary would — and found that **none of them gave a better privacy-versus-usefulness trade-off than simply blurring the data by hand**. Their paper is called *Synthetic Data — Anonymisation Groundhog Day*, and the title is the argument.

So: the naive fix fails, and the fashionable fix was not measured properly. That is the gap this project sits in.

2. What differential privacy actually is

Differential privacy is not a technique for hiding data. It is a **promise about what an answer can reveal**, and the promise is mathematical rather than procedural.

The coin-flip version, worked by hand

Suppose you want to survey a thousand people and ask: *"Have you ever cheated on an exam?"* Nobody will answer honestly, because the answer is embarrassing and you are writing it down.

So you change the rules. You hand each person a coin and tell them:

1. Flip the coin, secretly.
2. **Heads** — answer the question truthfully.
3. **Tails** — flip again. If the second flip is heads, say "yes". If it is tails, say "no".

Now think about what happens if someone says "yes". Did they cheat? You genuinely cannot tell. They might have flipped heads and told the truth. They might have flipped tails then heads and said "yes" for no reason at all. **Every person has a complete, honest excuse.** That excuse is called *plausible deniability*, and it is what differential privacy sells.

Now the useful part: you can still get the answer

Work out the two probabilities.

If someone really did cheat, they say "yes" when:

- they flip heads (probability 1/2) and tell the truth, **or**
- they flip tails then heads (probability 1/4).

So **P(says yes | really cheated) = 1/2 + 1/4 = 0.75**.

If someone really did not cheat, they only say "yes" when they flip tails then heads.

So **P(says yes | did not cheat) = 0.25**.

Now suppose you survey 1,000 people and, unknown to you, 300 of them really cheated.

- Of the 300 cheaters: 300 × 0.75 = **225** say yes.
- Of the 700 non-cheaters: 700 × 0.25 = **175** say yes.
- Total yeses you observe: **400**.

You do not know the 300. But you can recover it. You know each person says yes with probability 0.25 by pure noise, plus 0.5 extra if they are a real yes. So:

```
observed  =  0.25 × 1000  +  0.5 × (true number)
     400  =  250            +  0.5 × true
    true  =  (400 - 250) / 0.5  =  300
```

You got the population answer exactly right, and you learned nothing reliable about any individual. That is differential privacy in one page. Everything else is a more efficient version of this same trade.

What epsilon is

Look at those two probabilities again: 0.75 and 0.25. Their ratio is **3**.

That ratio is the whole guarantee. Whatever you observe, the explanation "this person cheated" is at most **3 times** more likely than "this person did not". Not a hundred times. Not certain. Three.

Epsilon is just that ratio written as a logarithm:

> e^ε = 3, so **ε = ln 3 ≈ 1.10**

That is the only maths you need. Now here are three ways to hold it in your head. Different ones click for different people — use whichever your listener responds to.

ε as a budget. You get one fixed allowance per dataset. Every question you ask spends some. When it runs out, you stop asking — not ask more carefully, *stop*. Small ε means a small allowance: few questions, or noisy answers.

ε as a noise dial. Turn it down and more noise is added: the data becomes safer and less useful. Turn it up and less noise is added: the data becomes sharper and less safe. There is no setting that gives you both, and being explicit about that trade is the point.

ε as a cap on any one person's influence. This is the formal one. If you removed *your* row from the database entirely and re-ran everything, the probability of any given output would change by at most a factor of e^ε. Your presence in the data cannot move the answer by more than that — which is exactly why nobody can work backwards from the answer to you.

Why smaller is safer, and why a billion is nonsense

There is a way to turn ε into a sentence a person can act on. If an attacker already knows every other record in the dataset and is trying to work out whether *you* are in it, the best they can do is:

> odds of a correct guess = e^ε / (1 + e^ε)

Run the numbers:

ε	best possible guess about you	in plain words
0.1	52 in 100	barely better than a coin flip
1.0	73 in 100	a real but bounded advantage
4.0	98 in 100	almost certain
8.0	100 in 100	no protection worth the name
1,000,000,000	100 in 100	the guarantee says nothing at all

This is not our invention. A 2023 study at USENIX Security tested how to communicate ε to 963 people and found that **odds-based explanations beat every alternative**, including showing the ε value on its own. Our tool prints this sentence on every release.

The last row is the one to remember. You will occasionally see a paper or a product claiming differential privacy at an enormous ε. The claim is technically true and practically empty: an ε that large permits publishing the raw data unchanged. **A DP guarantee without a stated ε — or with an absurd one — is marketing, not privacy.**

3. Synthetic data, and why "it's fake so it's safe" is false

Synthetic data means: look at a real table, learn its statistical shape, then generate a new table of people who do not exist but whose statistics match.

It sounds like it must be private. Nobody in the output is real. What could leak?

Two things, and both are common.

First: a generator can copy. Nothing in "generate a similar table" forbids reproducing a real row exactly. A model that has memorised an unusual patient — 96 years old, rare diagnosis, unusual postcode — may reproduce that person verbatim, because that record *is* the pattern it learned. The output is labelled synthetic. The row is a real human being.

We measured this in our own system before we fixed it. A generator running on a table with a medical record number column emitted **6 real MRNs verbatim at ε = 1, and 13 at ε = 8**. The file said synthetic. It contained real patient identifiers.

Second: statistics leak too. Even with no copying, if only one person in the dataset is a 92-year-old cardiologist, then a synthetic table that preserves the age-by-occupation relationship has told you that person exists. You did not need their row.

The sentence to say in the room

> "Looking plausible is not the same as being safe. You cannot see privacy by reading the output. That is why the rest of this project is about measuring it."

This is exactly why SynthProof exists. Synthetic data is a *delivery mechanism*. The privacy has to come from somewhere else — from a differentially private process that made the statistics before the generator ever saw them.

4. Our four generators, one paragraph each

All four take noisy counts and turn them into a table. None of them ever sees the real data.

Independent. Look at each column on its own — how many people are aged 20–30, how many 40–50, how many are in each job — add noise to those counts, then build the new table by picking each person's columns independently. Cheap, safe, and it destroys every relationship between columns. Someone's age tells you nothing about their income. It is our floor: if a cleverer method cannot beat this, it is not earning its budget.

Pairwise tree. Same idea, but also count *pairs* of columns: how many people are aged 20–30 **and** work part-time. Arrange those pairs into a tree so each column is linked to one other, and sample along it. This preserves some relationships. It costs more budget, because counting pairs is more questions.

AIM. The current state of the art for this kind of data, from a 2022 paper by McKenna and colleagues. It is adaptive: it spends a little budget working out *which* column pairs matter most for this particular dataset, then spends the rest measuring those. Being adaptive is what makes it good — and, as section 8 explains, it is also what made our benchmark misleading. **Important and say it unprompted: we did not write AIM.** We import it from the authors' `private-PGM` library. What we wrote is everything around it.

Moments. A deliberately weak control that fits a simple bell curve to each column separately. It exists so we have something obviously naive to compare against. Note for honesty: this was originally called "copula" in our code, which was wrong — a copula models the joint relationship and this does not. We renamed it. If someone asks about the name change, that is the answer.

5. The accountant and the ledger

The receipt metaphor, developed properly

Think of the dataset as a bank account with a fixed balance. The balance is ε. It does not refill.

Every question you ask of the real data is a withdrawal. Ask how many people are over 50 — that costs something. Ask for the age-by-income breakdown — that costs more, because it is a more detailed question. Ask a hundred small questions and you have spent as much as one large one. When the balance hits zero, **you stop.** Not "ask more carefully". Stop. The account is empty and any further answer breaks the promise you made to the people in it.

The **accountant** is the component that keeps the running total. Three things about ours are worth defending:

It bills the quiet questions. Most tools charge for the obvious query and forget the setup. Before you can count how many people are in each job category, you have to know what the job categories *are* — and if you learn that by looking at the data, that is a question about real people and it goes on the bill. Most tools read it for free. We charge it, or we require you to declare it publicly up front. This is a genuine difference and worth stating.

Generating is free, and that is a theorem. Once the counts have been made private, fitting a model to them and sampling a table costs nothing more. This is called *post-processing invariance*: any function of an ε-private output is still ε-private, because that function never saw the original data. It is one of the properties that makes DP usable at all.

We do not invent the arithmetic. Adding up privacy charges correctly is subtle and it is where implementations get it wrong. So we hand that job to Google's `dp_accounting` library and check our answers against a second, independent implementation. If someone wants to attack our accounting, the honest reply is: attack Google's library, because that is whose arithmetic it is. **A bound we cannot cite is a bound we do not print.**

The ledger

Every release appends a block: which dataset, which mechanism, how much ε it spent. Each block contains a fingerprint (a SHA-256 hash) of the block before it, so the blocks form a chain. Change any block and its fingerprint changes, so the next block's record of it no longer matches, and every block after that fails too. The whole chain is signed with a cryptographic key.

Say this exactly, and no more:

> "Tamper-evident against modification, reordering, and deletion of interior entries."

Then volunteer the interesting part, because it makes you look like you thought about it:

> "Chopping off the *end* of the chain is a different problem, and hash chaining alone does not catch it — a shortened chain is still perfectly consistent with itself, so an operator could delete the entries recording an overspend and still pass verification. We found that by attacking our own ledger. We fixed it with a signed head that commits to how long the chain is supposed to be."

Never say "tamper-proof". Anyone holding the signing key can rewrite history and re-sign it. Key custody is an organisational control, not a cryptographic one. If you are asked, say that — it is the correct answer and it is the one an expert is checking for.

6. Auditing: attacking our own release on purpose

The proof says the release is ε-private. But a proof is about the algorithm on paper. The thing that actually ran is code, and code has bugs. So we attack it.

What a canary is

A canary is a record we plant on purpose, so we know the answer.

Take the dataset. For each of *r* chosen records, flip a coin: heads, it goes in; tails, it stays out. Run the whole pipeline. Now look at the synthetic table and, for each of those *r* records, guess whether it was included.

If the release leaks nothing, your guesses are coin flips — you will be right about half the time. If the release leaks, you will do better than half. **How much better than half you can get, we convert into a number: ε_audited.**

The name comes from canaries in coal mines: a small thing you put in on purpose, which tells you something is wrong before it hurts anyone real.

The two numbers, and what each one is

– **ε_proved** is an **upper bound**. It says: the leak is *at most* this bad. It comes from

mathematics and holds against every possible attacker, including ones nobody has invented.

– **ε_audited** is a **lower bound**. It says: the leak is *at least* this bad. It comes from

a real attack, and it only speaks for the attacker we actually ran.

Both are useful and they answer different questions. Shipping both, on the same release, from one pipeline, is the thing we built.

Controls, and why they matter

Every audit runs against a control: a release with no planted records at all. If the auditor reports leakage there, it is measuring its own bugs rather than the mechanism.

We ran that control at seven canary counts — 10, 25, 50, 100, 200, 400, 800 — with five seeds each, so 35 runs. It reported zero in **34 of 35**. The one exception was a single seed at r = 100, which certified ε = 0.070.

That is not a bug, and you should say why. Our confidence threshold is α = 0.05, which means we accept a 5% chance of a false alarm by construction. One false positive in 35 runs is 2.9% — right where the maths says it should be. A control that reported *exactly* zero every single time would actually be more suspicious, because it would suggest the test was too conservative to detect anything. (Source: `results/detection_floor.json`.)

7. The ceiling — the hardest idea, and the one you must own

Read this section twice. If you can explain only one thing from this project, explain this. It is the actual research contribution, and it is a result *against our own headline*.

Where it starts

Our audits came back at **ε_audited = 0.000** almost everywhere. The lazy conclusion is "nothing leaked, we win". Before writing that down we asked a different question:

> **Could this instrument have detected anything, even if there had been something to detect?**

The answer was no. Here is why, and you can check it by hand.

The coin-flipping argument

You plant **r = 10** canaries. The attacker guesses in-or-out for each one, and gets **all ten right**.

Is that suspicious? Yes. If the release leaked nothing, each guess is a coin flip, so getting all ten right by luck has probability (1/2)^10 = 1/1024 ≈ 0.001. That is well under our threshold of 5%, so we can confidently say: something leaked.

Good. Now the crucial turn.

The turn

We are not testing "did anything leak". We are testing a **specific claim**: *this release is ε-private*. To reject that claim, the attacker has to do better than an ε-private release would allow.

An ε-private release *permits* the attacker to be right with probability:

> p = e^ε / (1 + e^ε)

— the same odds formula from section 2. So for our test to reject, we need the attacker's best possible performance, getting all *r* right, to be improbable *even under that permission*:

> **p^r <= α**, where α = 0.05 is our confidence threshold.

Now solve it for r = 10:

```
p^10  <=  0.05
   p  <=  0.05^(1/10)  =  0.741
   ε   =  ln( p / (1 - p) )  =  ln( 0.741 / 0.259 )  =  1.05
```

With 10 canaries, you can never certify an ε above about 1.05. Not with a better attacker. Not with a smarter statistical test. Not with more computing power. The limit is set by *r* alone, before a single row of data is generated.

Do it again for r = 30:

> p <= 0.05^(1/30) = 0.905, so ε <= ln(0.905 / 0.095) = **2.25**

And the formula in general:

> **ε_max(r) = log(a / (1 - a)), where a = α^(1/r)** — roughly log(r / ln(1/α))

Why this destroyed our headline

The ε we proved in our main experiment was **7.36**. Turn the formula around and ask how many canaries it would take to certify that:

ε you want to certify	canaries needed, with a perfect attacker
1.0	10
2.0	24
4.0	166
7.36	**4,711**
8.0	8,932

Roughly **e^ε** canaries — exponential. Certifying 7.36 would need about 4,700 canaries *every single one of which the attacker identifies correctly*. With a realistic attacker, far more. We ran 30 to 80.

So "ε_proved = 7.36, ε_audited = 0" is **not a finding about the mechanism.** It is a finding about the ruler. The gap cannot be closed by measuring harder, and reporting it as a result would have been wrong.

The punchline, which is genuinely useful

Look at what auditing *did* catch. When we deliberately broke a generator so it copied every record verbatim, the auditor caught it instantly at 10 canaries — attacker accuracy 100%, zero false alarms. When we made it copy only 5% of records, the auditor **never reliably detected it at any canary count we ran**. (Source: `results/detection_floor.json`.)

> **Auditing is a smoke alarm, not a thermometer. It catches broken implementations. It does not confirm that a correct one is tight.**

That is a genuinely practical conclusion for anyone deploying DP, and it is the sentence to finish on.

Credit — say this out loud, unprompted

This is not our theorem. The fact that empirical privacy lower bounds are limited by sample size is known in the literature: Kasiviswanathan & Smith (2014); Steinke, Nasr & Jagielski (NeurIPS 2023, §7); Nasr et al. (2023); Keinan, Shenfeld & Ligett (2025).

What is ours is narrower and still worth defending: the **empirical demonstration in the differentially private synthetic-tabular setting**, with the numbers measured rather than argued, and the engineering decision to print the ceiling next to every audited number so that nobody — including us — can misread a zero as a clean bill of health.

Claiming the theorem would be the easiest way to lose the room. Volunteering the citation is the easiest way to win it.

8. The confound, in plain language — RETRACTED

> **Read this first. We withdrew this claim on 2026-08-25.** A controlled experiment of our own (`results/SELECTION_ABLATION.md`, 175 fits) showed that AIM picks almost exactly the most strongly related pair of columns, so "pairs AIM selected" and "pairs that are actually related" are the same pairs. On the unrelated pairs the independent baseline is simply a correct model, not a weak one. The numbers below are real, but they do **not** show that AIM's advantage is an artefact of its own choice. Present this section only as an example of a reading we tested and dropped, never as a finding.

The one-liner (as originally stated, now withdrawn)

> **"We graded the exam using a question one of the students wrote."**

What happened

AIM is adaptive. Part of what makes it good is that it *chooses* which pairs of columns to model, spending a little budget on that decision.

Our fidelity score measured how well the synthetic data preserved the relationship between one specific pair of columns. On the UCI Adult dataset, that pair happened to be one AIM had chosen to model.

So we split the measurement in two: how does AIM do on pairs it picked, versus pairs it did not pick?

correlation error (lower is better)	AIM	independent baseline
on pairs AIM **selected**	0.026	0.112
on pairs AIM **did not select**	0.059	0.061

On the pairs it chose, AIM is about **four times better** than a generator that models no relationships between columns at all. On the pairs it did not choose, it is **the same as that generator**.

We first read this as "AIM's advantage lives exactly where it spent budget". The selection ablation shows the simpler explanation: AIM spends budget on the pairs that are actually related, and that is where any model of relationships can beat one that models none. (Sources: `results/clique_confound.json`, `results/SELECTION_ABLATION.md`.)

Why this matters beyond us

What survives is narrower: a fidelity score built on **one** column pair is fragile, whichever mechanism it favours. The fix is cheap — fix the set of column pairs in advance, chosen independently of any mechanism — and it is what we would do next.

The caveat you must volunteer

On our second dataset, ACS, the same test came back **inconclusive** — AIM selected no pair there, so the design cannot speak either way. Neither dataset supports the confound reading now. Say so. A result you have qualified yourself is much harder to attack than one a panellist has to qualify for you.

9. Our results, and what they do NOT mean

Three hypotheses, all registered in version control before any result existed.

H1 — do different mechanism families differ?

Correlation error at ε = 8, mean with 95% confidence intervals, five random seeds, 6,000 rows. Lower is better.

mechanism	UCI Adult	ACSIncome (California, 2018)
independent	0.0947 [0.0817, 0.1071]	0.0535 [0.0471, 0.0604]
pairwise tree	0.0283 [0.0132, 0.0517]	**0.0202 [0.0076, 0.0383]**
AIM	**0.0105 [0.0059, 0.0153]**	0.0621 [0.0405, 0.0837]

Sources: `results/h1_all_families.json`, `results/acs/h1_all_families.json`.

On Adult: supported. Both AIM and the pairwise tree beat the independent baseline — their intervals do not overlap with it — so that part is real and not noise. AIM has the lowest average, but its interval overlaps the pairwise tree's, so which of those two is best is not settled.

On ACS: the ordering inverts. Pairwise wins, and AIM stops being distinguishable from the baseline entirely.

What it does NOT mean: it does not mean AIM is bad, and it does not mean our first result was wrong. It means **the result did not transfer**, and section 8 gives a concrete reason why. Say this before anyone asks. Volunteering a failed replication is the single most credible thing you can do in a viva.

H2 — do minority subgroups leak more?

Not supported, on either dataset. 14 tests on Adult, 22 on ACS, and after correcting for testing many things at once, **zero** show significant leakage.

Do not say "we found nothing". Say this instead:

> "It is a **bounded** null. With 80 canaries at 5% confidence, the smallest effect we could have detected was ε = 0.008, the largest thing we could ever certify was 3.27, and the biggest effect we actually observed was 0.036 — about **1% of the instrument's range**. So we can say the effect, if it exists, is small. We cannot say it is zero."

Sources: `results/h2_analysis.json`, `results/acs/h2_analysis.json`.

The difference between a bounded null and a shrug is the whole project. A shrug says "we looked and saw nothing". A bounded null says "we looked, here is exactly how well we could see, and here is the largest thing that could have been hiding".

H3 — does spending more budget on important columns help?

Not supported, on either dataset. We declared which columns mattered, spent four times the budget on them, and downstream accuracy did not measurably improve at any of the five ε values. Source: `results/h3_allocation.json`.

What none of the results mean

– They do **not** mean this is safe for real patient data. It is not. There is no

cross-session budget enforcement, authentication is a single shared key rather than real identity, and only single tables are supported.

– They do **not** generalise beyond two datasets, five seeds, and one attacker. All three of our auditors use the same nearest-neighbour similarity score; a stronger, mechanism-aware attacker could move every number.

– ε_audited = 0 does **not** mean nothing leaked. See section 7. It means the instrument could not have seen it.

10. Glossary

Anonymisation — removing names and obvious identifiers. Provably insufficient (section 1).

AIM — an adaptive DP generator (McKenna et al., 2022) that spends budget choosing which column pairs to model. Imported from `private-PGM`; we did not write it.

Attacker / adversary — the hypothetical worst-case person trying to work out who is in the dataset. DP assumes they know everything except the one fact they are attacking.

Audit — attacking our own release on purpose to measure what actually leaks.

Bootstrap confidence interval — a range computed by resampling the data many times. If two intervals do not overlap, the difference is unlikely to be luck.

Canary — a record planted on purpose so we know the truth about it.

Ceiling (ε_max) — the largest ε a given number of canaries could *ever* certify, no matter how good the attacker. Section 7.

Composition — the rule for adding privacy charges together across many operations. Delegated to Google's `dp_accounting`.

Correlation error — how far the synthetic data's relationship between two columns drifted from the real one. Lower is better.

Delta (δ) — a tiny extra allowance for the guarantee failing outright. We use 1e-5: roughly a one-in-a-hundred-thousand chance.

Differential privacy (DP) — a guarantee that any one person's presence changes the output distribution by at most a factor of e^ε.

Epsilon (ε) — the privacy budget. Smaller is safer. Section 2.

FDR / Benjamini-Hochberg — a correction for testing many hypotheses at once. Without it, testing 22 things at 5% each means expecting about one false positive by chance.

Ledger — the hash-chained, signed record of every release and what it spent.

Marginal — a count. A one-way marginal counts one column; a two-way marginal counts a pair.

Membership inference attack (MIA) — an attack that tries to determine whether a specific person was in the training data.

Null result — a result where the expected effect was not found. A *bounded* null also states how large an effect could have hidden.

Post-processing invariance — anything computed from a private output is still private. Why generating costs nothing.

Privacy Data Sheet — the signed JSON file shipped with each release: both epsilons, the ceiling, which domains were declared versus inferred, and known residual risks.

Seed — the fixed number that makes a random run repeat exactly. Every result we report names its seeds.

Synthetic data — a generated table matching the statistics of a real one. Not automatically private (section 3).

Tamper-evident — an alteration cannot be made silently. *Not* the same as tamper-proof.

TSTR (Train on Synthetic, Test on Real) — train a classifier on the synthetic data, test it on real data. Measures whether the synthetic data is actually useful.

11. The Q&A drill

Hardest first. Learn the shape of each answer, not the words.

1. What is actually new here? Isn't this just SDV plus a logger? The integration is engineering; DP synthesis and one-run auditing both already exist. Our contribution is two negative results about the measurement instruments: the audit ceiling demonstrated empirically in the DP synthetic-tabular setting, which disqualifies the proved-versus-audited comparison at these budgets, and evidence that a standard fidelity metric measures an adaptive mechanism's own choice of column pair. SDV does not ship an ε at all, let alone two, and does not report the limits of its own measurement.

2. Your H2 found nothing. So what? It is a bounded null, not an absence. Largest effect observed 0.036 against an instrument range of 3.27 — about 1% used. We can bound the effect; we cannot claim it is zero, and we say so.

3. Why should I trust your epsilon? Don't trust ours — we didn't write it. Composition goes through Google's `dp_accounting` and is differentially tested against a second independent implementation. Calibration is conservative: request ε = 1 and we deliver 0.912, never more. The noise samplers are verified against theory to within 0.3%.

4. Did you write private-PGM or import it? Imported. AIM is McKenna et al.'s work. We wrote the accountant integration, the refusal gate, the auditor, the ledger, the data sheet, and every experiment.

5. What breaks if I upload my own data? Quite a lot, on purpose. Single table only. It refuses identifier columns, free text, tables under 500 rows, tables with no categorical column, and column pairs whose grid is too large. You must declare a schema or pay ε to discover one — because understanding an unknown table is itself a query against sensitive data. SmartNoise and Tumult require the same thing.

6. What is your weakest result? H1's structure finding. It held on Adult with non-overlapping intervals and inverted on ACS. Section 8 gives a mechanism for why, but that explanation is itself only confirmed on one dataset. It needs a third dataset before it is more than a hypothesis.

7. So your headline comparison is dead? At these ε values, yes, and we say so in the thesis. Reporting "proved 7.36, audited 0" as a finding about a mechanism would have been wrong. Finding that out and reporting it is the result.

8. Is the ceiling your theorem? No. Kasiviswanathan & Smith 2014; Steinke, Nasr & Jagielski NeurIPS 2023 §7; Nasr et al. 2023; Keinan, Shenfeld & Ligett 2025. Ours is the empirical demonstration in this setting and the decision to report the ceiling beside every audited number.

9. Is this tamper-proof? No. Tamper-evident. Anyone holding the signing key can rewrite and re-sign. Key custody is an organisational control.

10. Can I use this on real hospital data? No, and the README says so. No cross-session budget enforcement, authentication is one shared key, single table only. It is an instrument for checking DP releases, not a production pipeline.

11. Why only two datasets? Time. One was the original plan; a single-dataset conclusion is weak, so we added ACS — which is what surfaced the contradiction. A third is the highest-value next step.

12. Why five seeds? Five seeds across three mechanisms and five ε values is 75 runs per dataset, and the ACS grid alone took 4,600 seconds. We report bootstrap intervals so you can see the uncertainty rather than having to trust the mean.

13. Where is the diffusion model you promised in the synopsis? Dropped, deliberately. It is VRAM-bound and the available hardware is a 4 GB RTX 3050, where DP-SGD per-sample gradients run out of memory.

More importantly it was not needed: H1 asks whether mechanism *families* differ, and independent, pairwise-tree and AIM are already three distinct families. It is our largest deviation from the synopsis and it is declared.

14. Where is the shadow-model membership attack? Not built. It needs 64 or more shadow *generators* — days of compute for a result that is noisy on tabular data at this scale. Declared as future work. We did not rename a weaker attack to sound like it: our distance-based attack is labelled a weak baseline, not LiRA.

15. What about Celery, Redis, MinIO, DuckDB, Postgres, multi-table? None of them exist. Each was dropped because none had an experiment behind it. Adding infrastructure would have consumed the budget that produced the ceiling result.

16. You found twelve bugs in your own code. Should I trust the rest? Twelve found is evidence of looking. The questions that matter are whether we found them or a reviewer did, and whether each has a test that fails without the fix. Both answers are yes, and the tests are named after the defects.

17. What was the worst bug? A profiler that returned the true most-common category when every noisy count fell below threshold — in 196 of 200 seeds at a small budget. A privacy tool silently emitting real data. Fixed, with a regression test that fails on the old code.

18. How do I know your numbers are real? `make reproduce` regenerates every result file from a committed manifest at fixed seeds and checks the hashes. It currently passes. No number in the thesis may cite a value that is not in a committed results file.

19. Isn't a null result a failed project? All three hypotheses were registered before any result existed, and every deviation is declared in a table. Two nulls and one non-replication, each with a stated detection floor, is a more honest outcome than three confirmations with no power analysis.

20. Why is ε_proved lower than what I requested? Calibration searches by bisection and returns the conservative end of the bracket. Under-spending is safe; over-spending breaks the guarantee. An earlier version of this code requested 8 and delivered 70 — that failure is why the bracket is conservative.

21. Are you immune to floating-point side-channel attacks? No, and we do not claim to be. We use discrete Gaussian and discrete Laplace samplers rather than rounding floats, which removes the best-known class of such attacks, but we have not done a timing analysis and would not claim immunity.

22. Your attacker is weak. Wouldn't a better one change everything? It could raise every audited number, and the ordering might change. All three of our auditors share a nearest-neighbour similarity score, which we state as a threat to validity. Note the ceiling is unaffected — a better attacker cannot exceed it.

23. Why does generating the synthetic table cost no privacy? Post-processing invariance. Any function of an ε-DP output is still ε-DP, because it never touched the original data.

24. What is delta and why can you ignore it? δ is a small probability that the guarantee fails outright. We use 1e-5. In the audit it is handled by a union bound rather than the tighter treatment in the paper — conservative rather than exact, and at this δ the correction is negligible.

25. Why did you rename "copula" to "moments"? Because it was not a copula. It fits per-column Gaussian moments with no covariance and no rank transform. Calling it a copula overstated what it does, so we renamed it in code and in every document.

26. What would you do with another six months? In order: pre-register column pairs chosen independently of any mechanism, to fix the confound; add a third and fourth dataset to find out which ordering is the accident; build a stronger mechanism-aware attacker; then cross-session budget enforcement.

27. What is the single sentence summary? The synopsis described a platform. What we built is a calibrated instrument, plus the finding that its working range does not cover the claim it was built to check.

12. Cheat sheet — one page, take it in with you

The one-line pitch. Synthetic data that ships with two numbers: what we proved mathematically, and what an attacker actually recovered — plus the limits of the measurement itself.

The opening line. "A hospital has the data that would answer your question, and it cannot give it to you. So it strips the names. That has been the standard answer for thirty years and it does not work."

Epsilon in one sentence. A privacy budget. Smaller is safer. At ε = 1 an attacker who knows everything else can guess whether you are in the data about 73 times in 100; at ε = 8, 100 in 100.

The four things to never say.

1. "Tamper-proof" — say tamper-*evident*.
2. "The ceiling is our theorem" — cite Steinke et al. 2023 §7 and Kasiviswanathan & Smith 2014.
3. "Nothing leaked" — say the auditor found nothing above its floor, and state the ceiling.
4. "It's synthetic so it's safe" — that is the belief the project exists to disprove.

The numbers to know cold.

H1, AIM vs independent, Adult, ε=8	0.0105 vs 0.0947, intervals separate
H1 on ACS	ordering inverts; AIM stops separating
H2	14 and 22 tests, 0 survive FDR; observed 0.036 of a 3.27 range
H3	not supported, both datasets
ceiling at r = 30	2.25, against a proved ε of 7.36
canaries to certify 7.36	~4,711, all correct
leak detection	100% copying caught at r=10; 5% never caught
tests / coverage	436 passing, 94%
confound, Adult	selected pairs 0.026 vs 0.112; unselected 0.059 vs 0.061

The three things to volunteer before being asked.

1. H1 did not replicate on the second dataset — and here is the mechanism why.
2. The ceiling disqualifies our own headline comparison.
3. AIM is imported, not written by us; diffusion and shadow-model MIA were dropped.

If you lose the thread. Go back to: *this project is about whether the privacy claim is true, and most of what it does is refuse to overclaim.* That sentence is true of every slide.

Closing line. "A smaller system than we promised, and a larger result than we expected." Then stop.

Measurement convention — the ceiling is borrowed, and attributed

The audit ceiling reported beside every ε_audited is `log(r / ln(1/α))`, a one-line corollary of Steinke, Nasr & Jagielski (NeurIPS 2023, arXiv:2305.08846) Thm 2.1 — **not a result of ours** — and the same quantity is already named *maximum auditable epsilon* by Annamalai, Ganev & De Cristofaro (USENIX Sec 2024, arXiv:2405.10994) §2.2.

Reporting it alongside the measurement is a transfer of **limit-of-detection (LoD) reporting** from analytical chemistry, where **MIQE 2.0** (Bustin et al., *Clinical Chemistry* 2025;71(6):634–651) mandates LoD/LLOQ disclosure and a laboratory reports *"Not Detected, < LOD"* rather than zero. The transfer is the claim; the

convention is not our invention.

Full attribution: [`docs/MEASUREMENT_CONVENTIONS.md`](../MEASUREMENT_CONVENTIONS.md).